

RUBRIC-FEEDBACKRAPPORT: OpenMRS LegacyUI v1.20.0

Gegenereerd: 2026-06-17 | Auditor: Claude Sonnet 4.6 | Repo: legacyui-audit (branch: main)

1. SAMENVATTINGSTABEL

ID	Criterium	Status	Max	Niveau	Korte bevinding
F0.1	SonarCloud config	ONTBREEKT	-	N.v.t.	Geen sonar-project.properties of pom.xml sonar-plugin gevonden
F0.2	SonarCloud workflow	ONTBREEKT	-	N.v.t.	Enkel CodeQL + Snyk + JaCoCo-CI; geen SonarCloud-scan workflow
F0.3	Quality gate enforcement	ONTBREEKT	-	N.v.t.	Geen SONAR_TOKEN of qualitygate.wait=true in enige workflow
F0.4	Gate-keuze gedocumenteerd	ONTBREEKT	-	N.v.t.	Geen beslisdocument Clean as You Code vs baseline gate
C1	Analyse onderhoudbaarheid	AANWEZIG	20	Goed	Alle 5 ISO 25010-subkenmerken, CC-tabellen, coverage-baseline, prioriteringstabel
C4	Aangepast ontwerp	AANWEZIG	20	Goed	Strategy+registry, 3 gewogen alternatieven, motivatie op OCP/ISO 25010
C2	Testopzet & testresultaten	GEDEELTELIJZIG	20	Voldoende	CFG+basispaden+15 TC aanwezig; testresultaten-sectie is placeholder
C3	Verbeteringen prioritering	AANWEZIG	10	Goed	Impact x effort, 9 voorstellen, bronverwijzingen C1/C5/C6
C5	Realisatie PoC & verantwoording	AANWEZIG	10	Goed	12 strategy-klassen op main; kritische AI-reflectie aanwezig
C6	Validatie testen & regressietesting	GEDEELTELIJZIG	20	Voldoende	CC voor/na (handmatig), 399-tests bewijs; geen SonarCloud gate

2. KRITIEKE ONTBREKENDE ITEMS (ONTBREEKT)

SonarCloud/quality-gate tooling ontbreekt volledig. Dit is de grootste tekortkoming van het gehele beroepsproduct.

[F0.1] SonarCloud-project configuratie

Verwacht pad: sonar-project.properties (root) of sonar-plugin in pom.xml

Impact op rubric: Zonder projectconfiguratie kan SonarCloud de module niet identificeren en analyseren. Alle F0.x-criteria zijn daarmee 0/4 niet-haalbaar.

[F0.2] GitHub Actions workflow voor SonarCloud-scan

Verwacht pad: .github/workflows/sonar.yml met SonarSource/sonarcloud-github-action of sonar-scanner

Impact op rubric: CI-pipeline mist de centrale kwaliteitsgateway. CodeQL en Snyk zijn aanwezig maar geen SonarCloud. F0.2 is volledig afwezig.

[F0.3] SONAR_TOKEN secret en sonar.qualitygate.wait=true

Verwacht pad: .github/workflows/sonar.yml (SONAR_TOKEN als repo-secret, qualitygate.wait=true in scan-stap)

Impact op rubric: Zonder quality gate enforcement blokkeert de pipeline niet bij kwaliteitsachteruitgang. Het is onmogelijk te bewijzen dat de gate groen was.

[F0.4] Beslisdocument over toolingkeuze (Clean as You Code vs baseline)

Verwacht pad: Sectie in docs/onderhoudbaarheid/analyse-onderhoudbaarheid.md §0, of apart sonar-keuze.md

Impact op rubric: De verantwoording van de metriekkeuze (waarom JaCoCo ipv SonarCloud, of waarom Clean as You Code) ontbreekt volledig. Dit raakt ook de motiveringseis.

[C1.4] SonarCloud-gemeten CC-waarden met McCabe-banden als toolingbewijs

Verwacht pad: Sectie 4 van analyse-onderhoudbaarheid.md: CC-tabel met SonarCloud-waarden (niet schattingen)

Impact op rubric: CC-waarden zijn handmatig geschat (consistent maar niet via tooling gemeten). Rubric C1.4 vereist SonarCloud-meting. De handmatige telling is goed onderbouwd maar mist het tool-bewijs dat de rubric als eis stelt.

[C6.4] SonarCloud quality gate rood->groen + commit-hash/scandatum

Verwacht pad: docs/validatie.md sectie 6 of 7: SonarCloud-scan-URL, datum, of screenshot

Impact op rubric: Validatie gebruikt JaCoCo-getallen maar geen SonarCloud gate-bewijs. C6.4 is niet aantoonbaar. GOED-niveau voor C6 (C6.7) vereist sterk onderbouwd en reproduceerbaar bewijs.

[C5.1] Aparte feature-branch voor de PoC (gescheiden van de gemeten baseline)

Verwacht pad: feat/onderhoudbaarheid-formattag-strategy of feature/format-tag-strategy als actieve remote branch

Impact op rubric: Strategy-klassen zijn gemerged naar main (commit a3942bb). De branch feat/onderhoudbaarheid-formattag-strategy wordt in realisatie-poc.md en validatie.md vermeld maar bestaat niet als actieve lokale branch (git branch -a toont haar niet). Hierdoor is de scheiding baseline vs PoC onduidelijk. Rubric C5.1 stelt een separate branch als eis.

3. GEDEELTELIJKE ITEMS (GEDEELTELIJK)

C2 — Testresultaten nog niet ingevuld in testopzet.md

Wat er is: testopzet.md sectie 5 bevat letterlijk de placeholder "In te vullen na uitvoering (voor de refactor)". De 15 TC zijn geïmplementeerd in FormatTagTest.java (commit a3942bb) en validatie.md meldt 399 tests 0 failures, maar de per-TC baseline-run (rood/groen voor de refactor) is niet gedocumenteerd in het C2-document zelf.

Wat nog nodig / drempel geblokkeerd: C2.6 (testresultaten gedocumenteerd) en C2.8 (GOED: reproduceerbare resultaten). Zonder ingevulde resultaten is C2 maximaal Voldoende.

C2 — CFG-referentie naar extern PDF-bestand, niet inline visueel diagram

Wat er is: testopzet.md sectie 2 verwijst naar "img/printobject-cfg.png" maar het bestand is printobject-cfg.pdf (docs/img/). De CFG-knooppunten zijn uitgeschreven als tabel in de markdown (D1-D13 + beslispunten), maar een visueel diagram ontbreekt in het document zelf.

Wat nog nodig / drempel geblokkeerd: C2.1 (controlflow graaf). Functioneel aanwezig via tabel; visueel diagram alleen als extern PDF.

C6.2 — Coverage "voor" is schatting uit C1, niet een gemeten JaCoCo-baseline

Wat er is: validatie.md sectie 5 toont "Branch coverage voor: 40% (65/162)". Dit getal komt uit de schattingen van analyse-onderhoudbaarheid.md, niet uit een JaCoCo-run op het baseline-commit. De "na"-meting (35/110) is wel een JaCoCo-meting. Vergelijking is asymmetrisch.

Wat nog nodig / drempel geblokkeerd: C6.2 (JaCoCo voor/na) en C6.7 (GOED: sterk onderbouwd en reproduceerbaar). Een gemeten baseline is nodig voor de GOED-drempel.

C4.1 — UML-diagram als extern PDF, niet inline leesbaar

Wat er is: aangepast-ontwerp.md verwijst naar docs/img/aangepast-ontwerp-diagram.pdf. Het document bevat code-fragmenten die de structuur tonen (FormatStrategy interface, concrete strategy-klassen, registry). Het PDF-diagram bestaat maar is niet als PNG embedded in de markdown.

Wat nog nodig / drempel geblokkeerd: C4.1 (UML-diagram huidige situatie). Aanwezig als PDF; beoordelaar moet extern bestand openen.

C6.1 en C6.5 — CC voor/na is handmatige telling, niet SonarCloud-meting

Wat er is: validatie.md sectie 3: "CC na = 4 (handmatig geteld op werkelijke code)". Dit is methodologisch correct (consistent met C1-methode) maar mist het SonarCloud-toolingbewijs dat F0 en C6.4 vereisen. De kwantificeerbare verbetering (CC daalt van ~13-15 naar 4) is aantoonbaar aanwezig, maar de tool-reproducibiliteit ontbreekt.

Wat nog nodig / drempel geblokkeerd: C6.1 (SonarCloud CC voor/na) en C6.5 (kwantificeerbaar aantoonbaar). Handmatige telling is aanwezig; SonarCloud-bewijs ontbreekt.

4. STERKE PUNTEN (AANWEZIG)

C1 — Diepgaande analyse met alle 5 ISO 25010-subkenmerken en concrete code-referenties

analyse-onderhoudbaarheid.md behandelt Modularity, Reusability, Analyzability, Modifiability en Testability elk met exacte regelnummers, methode-namen en klasse-namen (bijv. PersonFormController.updatePersonAddresses() r. 543-709, CC 76). De metriekketen (kwaliteitseigenschap -> subkenmerk -> metriek -> meetwaarde -> interpretatie) is als tabel opgesteld met de correcte disclaimer dat metriek indicatief is, geen doel. Scope-afbakening (org/openmrs/web/** vs org/springframework/**) is expliciet. CC-tabellen zijn nauwkeurig opgebouwd met beslispuntentelling per groep.

C4 — Grondig ontwerpdocument met drie gewogen alternatieven en kwaliteitsmotivatie

aangepast-ontwerp.md beschrijft de smell (SRP+OCP-schending in printObject()), het Strategy+registry-ontwerp (FormatStrategy interface, FormatContext waardeobject, FormatStrategyRegistry), en verwerpt drie alternatieven: Visitor (vereist accept() op core-domeinklassen, buiten scope), kale lambdas (testbaarheidsgebrek), niets doen (lost OCP niet op). Motivatie is expliciet op ISO 25010-subkenmerken en Open/Closed Principle. De PoC-scope is bewust afgebakend en gemotiveerd.

C5 — PoC volledig gerealiseerd met kritische AI-reflectie boven beschrijvend niveau

realisatie-poc.md toont 12 strategy-klassen + registry + context (allemaal aanwezig op main, commit a3942bb). De AI-verantwoording (sectie 5) beschrijft 4 gecontroleerde fasen en evalueert kritisch: welke beslissingen NIET aan de tool zijn overgelaten (volgorde vangnet eerst, nul aanpassingen aan bestaande tests geverifieerd via git diff, CC-schatting van tool geverifieerd tegen werkelijke code). Dit is evaluatief, niet louter beschrijvend.

C3 — Expliciete impact x effort prioritering met P1/P2/P3 en bronverwijzingen naar C1

verbeterbacklog.md heeft een scoringsmethodiek (impact/effort 1-5, P1/P2/P3), 9 verbeteringsvoorstellen elk met bron-C1-verwijzing, motivatie en relatie tot C5/C6. De keuze om M1 (CC 76, updatePersonAddresses) NIET als PoC te kiezen is onderbouwd: 0% dekking + latente NPE maakt regressie-baseline onbetrouwbaar. Dit toont methodologisch inzicht.

C2 — 15 basispaden ontworpen met TC-tabel, geïmplementeerd in FormatTagTest

testopzet.md heeft CFG-tabel (D1-D14 beslispunten, CC=13), basispaden P1-P15 met route en beschrijving, en een TC-tabel met invoer/verwachte uitkomst/pad/bestaand-vlag. FormatTagTest.java (commit a3942bb) implementeert TC1/TC3/TC4/TC11/TC13/TC15 plus de caseConversion=global-case. Twee testtypen worden gecombineerd (white-box path coverage + black-box via doStartTag).

C6 — Regressie-aanpak methodologisch sterk: git diff als bewijs van test-integriteit

validatie.md meldt 399 tests, 0 failures en documenteert de git-diff-controle (144 toevoegingen, 0 wijzigingen aan bestaande tests) als bewijs dat geen bestaande assertie is aangepast. De nuancering van de coverage-daling (FormatTag % daalt maar absoluut aantal branches daalt ook: 162->110) is methodologisch volwassen en eerlijk.

Implementatie-kwaliteit strategy-code

FormatStrategyRegistry.java gebruikt LinkedHashMap in cascade-volgorde zodat specifieke typen (Provider, Form, Program die ook OpenmrsMetadata implementeren) voor de generieke MetadataFormatStrategy worden gevonden. De resolve()-methode hanteert eerst exacte match (voor Hibernate-proxy-correctheid) dan assignable match. FormatContext isoleert tag-state zodat strategies zonder JSP-context testbaar zijn. Bewuste afwijking van ontwerp (JavaScriptEscape niet in FormatContext) is correct verantwoord.

5. RUBRIC-SCORE SCHATTING

Let op: SonarCloud ontbreekt volledig. De scoring hieronder gaat ervan uit dat de docent de handmatige CC-metingen en JaCoCo-coverage als voldoende bewijs accepteert voor de kwaliteitsmetingseis. Als SonarCloud als harde eis wordt behandeld, dalen C1 en C6 mogelijk naar een lager niveau.

Criterium	Max	Niveau	Punten	Motivatie
Fase 0 (SonarCloud)	—	GEBLOKKEERD	0/4 checks	Geen sonar-project.properties, geen SonarCloud-workflow, geen quality gate, geen keuzedoc.
C1 — Analyse onderhoudbaarheid	20	Goed	16-18	Alle 5 subkenmerken, concrete code-refs, CC-tabellen nauwkeurig opgebouwd, metrieketen n
C4 — Aangepast ontwerp	20	Goed	17-19	Smell expliciet benoemd (SRP+OCP), Strategy+registry ontwerp met interface en handlers, 3 a
C2 — Testopzet & testresultaten	20	Voldoende	14-16	CFG + CC + basispaden + TC-tabel aanwezig; tests geïmplementeerd in FormatTagTest.java.
C3 — Verbeteringen prioritering	10	Goed	8-9	Impact x effort matrix, P1/P2/P3, 9 voorstellen, bronverwijzingen C1/C5/C6. Methodologische n
C5 — Realisatie PoC & verantwoording	10	Goed	8-9	PoC volledig gerealiseerd (12 strategy-klassen), ontwerp gevolgd, kritische AI-reflectie aanwe
C6 — Validatie testen & regressie	20	Voldoende	14-16	CC voor/na (handmatig, CC 13->4), coverage voor/na (deels JaCoCo), 399-tests-0-failures reg
TOTAAL	100	—	77-87	Ruimschoots boven de drempel van 55/100. Risicoscenario (SonarC

Drempel 55/100: WAARSCHIJNLIJK RUIMSCHOOTS GEHAALD (schatting 77-87/100)

Kanttekening: als docent SonarCloud als harde eis behandelt, kan score dalen naar ~60-70. Nog steeds boven drempel, maar GOED-niveau voor C1 en C6 staat dan onder druk.

6. GENUMMERDE ACTIELIJST (gesorteerd op urgentie)

Actie 1 [F0.1 t/m F0.4, C1.4, C6.1, C6.4] -- **CRITICAL**

Actie: Voeg SonarCloud toe: (a) maak sonar-project.properties aan in de root, (b) voeg .github/workflows/sonar.yml toe met SONAR_TOKEN secret en sonar.qualitygate.wait=true, (c) voer een scan uit en documenteer de CC-waarden in analyse-onderhoudbaarheid.md sectie 4 (vervang schattingen door SonarCloud-waarden), (d) leg de gate-keuze (Clean as You Code) vast.

Verwacht pad: sonar-project.properties (root); .github/workflows/sonar.yml;
docs/onderhoudbaarheid/analyse-onderhoudbaarheid.md sectie 4

Actie 2 [C5.1] -- **HIGH**

Actie: Herstel de feature-branch-eis: de PoC-code (Strategy-klassen) staat nu op main (commit a3942bb). Verifieer of remote branch feat/onderhoudbaarheid-formattag-strategy nog bestaat op GitHub (niet in lokale git branch -a). Als die branch weg is: documenteer in realisatie-poc.md sectie 1 de commit-hash (a3942bb) als PoC-referentie en leg uit waarom de branch naar main is gemerged en waar de originele branch-naam vandaan komt. Of maak een archief-branch aan op dat commit als bewijs.

Verwacht pad: docs/realisatie-poc.md sectie 1; remote branch feat/onderhoudbaarheid-formattag-strategy

Actie 3 [C2.6, C2.8] -- **HIGH**

Actie: Vul de testresultaten in testopzet.md sectie 5 in: voer mvn -B test -pl omod -Dtest=FormatTagTest uit op de baseline-code (commit voor a3942bb) en noteer per TC de uitkomst (groen/rood). Dit is het baseline-bewijs dat de testcases correct waren voor de refactor. Voeg ook de gemeten branch-coverage van printObject() toe als baseline-getal.

Verwacht pad: docs/testopzet.md sectie 5 -- verwijder placeholder; voeg tabel toe met per TC: status (groen/rood), commit-hash, branch-coverage printObject() voor de refactor

Actie 4 [C6.2, C6.7] -- **HIGH**

Actie: Voeg een echte JaCoCo-baseline-meting toe voor de coverage-vergelijking: git checkout && mvn -B test -pl omod en lees omod/target/site/jacoco/ uit. Noteer exacte branch-cijfers voor FormatTag (niet de schatting 65/162) in validatie.md sectie 5. Voeg commit-hash toe.

Verwacht pad: docs/validatie.md sectie 5 -- vervang "40% (65/162) - schatting" door gemeten JaCoCo-waarden; voeg commit-hash en meetdatum toe

Actie 5 [C4.1] -- MEDIUM

Actie: Maak UML-diagrammen direct leesbaar: exporteer aangepast-ontwerp-diagram.pdf naar PNG en embed in aangepast-ontwerp.md. Minimaal een class-diagram met FormatStrategy (interface), concrete handlers (ConceptFormatStrategy, UserFormatStrategy, etc.), FormatContext en FormatStrategyRegistry met relaties (implements, uses, contains).

Verwacht pad: docs/aangepast-ontwerp.md sectie 3; docs/img/aangepast-ontwerp-diagram.png

Actie 6 [C6.4] -- MEDIUM

Actie: Documenteer SonarCloud gate-overgang zodra actie 1 is uitgevoerd: voeg in validatie.md sectie 6 een SonarCloud-scan-URL of screenshot toe als evidence. Als SonarCloud niet wordt toegevoegd: voeg een expliciete verantwoording toe waarom JaCoCo + handmatige CC als equivalent bewijs wordt beschouwd.

Verwacht pad: docs/validatie.md sectie 6 of 7 -- SonarCloud-scan-URL, datum, of verantwoording

Actie 7 [F0.4] -- MEDIUM

Actie: Documenteer de toolingkeuze: schrijf een sectie in analyse-onderhoudbaarheid.md over waarom JaCoCo (ipv SonarCloud) is gebruikt, of voeg SonarCloud toe met de keuze voor Clean as You Code gate en documenteer die keuze. Zonder toolingverantwoording mist de motiveringseis.

Verwacht pad: docs/onderhoudbaarheid/analyse-onderhoudbaarheid.md -- voeg sectie 0 "Toolingkeuze" toe of integreer in sectie 3 Metriekketen

AANVULLENDE TECHNISCHE AANTEKENINGEN

Implementatiekwaliteit strategy-klassen:

FormatStrategy.java, FormatStrategyRegistry.java en FormatContext.java zijn technisch goed uitgewerkt. LinkedHashMap in cascade-volgorde, exact+assignable resolve() voor Hibernate-proxy-correctheid, FormatContext voor test-isolatie. De bewuste afwijking van het ontwerp (JavaScriptEscape niet in FormatContext na verificatie) is correct verantwoord.

FormatTagTest uitbreiding (commit a3942bb):

TC1/TC3/TC4/TC11/TC13/TC15 plus caseConversion=global-test zijn geïmplementeerd, exact zoals testopzet.md voorschreef. De eerder gerapporteerde gaten (provider, SingleCustomValue, lege collectie, datum, global CC) zijn gedicht.

Scope-keuze PoC (FormatTag boven PersonFormController CC 76):

De onderbouwing in aangepast-ontwerp.md sectie 1 waarom FormatTag boven de zwaarste methode is gekozen (0% dekking + latente NPE maakt betrouwbare baseline onmogelijk) is methodologisch overtuigend. Dit toont inzicht dat onderhoudbaarheid verder reikt dan simpelweg de hoogste CC aanpakken.

Ontbrekende lokale feature-branch:

realisatie-poc.md en validatie.md noemen branch "feat/onderhoudbaarheid-formattag-strategy" maar git show-ref toont deze niet lokaal. De strategy-code staat op main (commit a3942bb). De remote branch docs/onderhoudbaarheid-c2-testopzet en feature/sprint3-coverage bestaan nog wel als remote branches. Dit is de grootste C5.1-risicofactor.

CI-pipeline kwaliteit (niet-SonarCloud):

ci.yml (JaCoCo + SBOM), codeql.yml (CodeQL SAST) en snyk-sca.yml (Snyk SCA) zijn aanwezig en goed geconfigureerd. JaCoCo-rapport wordt als CI-artifact gepubliceerd. Dit is een sterke CI-opzet; SonarCloud is het enige ontbrekende element.